

Affixa — Morphological Analyzer

Solo project — developed without teammates.

Overview

Affixa is a web application for morphological analysis of English words. It identifies and analyzes prefixes, roots, and suffixes, providing educational feedback and scoring against gold-standard datasets. The application consists of a React + Vite frontend and a Python FastAPI backend, containerized with Docker for consistent deployment.

Architecture

Component	Technology	Port
Frontend	React, Vite, TypeScript, TailwindCSS	3001 (host)
Backend	Python 3.11, FastAPI, uvicorn	8001 (host)
Authentication	Supabase (OAuth + email magic links)	—
Dictionary data	JSON files (<code>prefixes.json</code> , <code>suffixes.json</code>)	Loaded at build time

Data Flow

1. User submits a word via the frontend
2. Frontend sends the word to `/api/analyze` (backend)
3. Backend uses `MorphologicalAnalyzer` to parse the word
4. Results are returned and displayed with highlights
5. Optional: comparison against gold-standard labels

Key Files

File	Purpose
frontend/src/App.tsx	Global recovery link forwarder (/reset-password catch-all)
frontend/src/pages/auth/ForgotPassword.tsx	Rate-limited forgot-password with 60s cooldown
frontend/src/pages/auth/ResetPassword.tsx	PKCE code exchange on mount; expired-link fallback
frontend/src/lib/supabase.ts	Supabase client with detectSessionInUrl
backend/app/main.py	uvicorn entry point on 0.0.0.0:8000
backend/app/nlp/analyzer.py	Core morphological analysis logic
backend/app/nlp/dictionaries/prefixes.json	24 prefix entries
backend/app/nlp/dictionaries/suffixes.json	24 suffix entries
backend/requirements.txt	Python dependencies (fastapi, uvicorn, spacy, nltk, supabase, etc.)
frontend/Dockerfile	Node 22 build → nginx + SPA fallback
backend/Dockerfile	Python 3.11-slim + WordNet pre-download
docker-compose.yml	Ports 8001 ↔ 8000, 3001 ↔ 80; VITE_API_URL=http://localhost:8001/api
docs/DEPLOYMENT.md	Deployment checklist + auth redirect notes

Setup & Local Development

⚠ Host ports 3000, 5173, and 8000 are occupied (MarketMind, LINEAGE, WSL relay). Containers use **3001** and **8001** instead.

Prerequisites

- Docker and Docker Compose
- Node.js 22 (for local frontend dev, not required if using Docker only)
- Python 3.11 (for local backend dev, not required if using Docker only)

Using Docker (recommended)

```
docker compose up -d --build
```

- Frontend: `http://localhost:3001`
- API: `http://localhost:8001/api/...`
- Health check: `http://localhost:8001/api/health` → `{"status": "ok", "service": "affixa-api"}`

Local development (without Docker)

Backend:

```
cd backend
python -m venv venv
.\venv\Scripts\activate
pip install -r requirements.txt
python -c "import nltk; nltk.download('wordnet', quiet=True)"
python app/main.py
```

Frontend:

```
cd frontend
npm install
npm run dev
```

Open `http://localhost:5173` (or as Vite reports). Set `VITE_API_URL=http://localhost:8000/api` in `.env` if the backend runs locally.

API Reference

Endpoint	Method	Description
<code>/api/health</code>	GET	Returns { "status": "ok", "service": "affixa-api" }
<code>/api/analyze</code>	POST	Analyze a word: { "word": "string" } → { prefix, root, suffix }
<code>/api/compare</code>	POST	Compare analysis against gold-standard CSV
<code>/api/batch</code>	POST	Analyze a batch of words: { "words": ["string", ...] }

Example: `/api/analyze`

```
curl -X POST http://localhost:8001/api/analyze \  
  -H "Content-Type: application/json" \  
  -d '{"word": "unfortunately"}'
```

Response:

```
{  
  "prefix": "un",  
  "root": "fortunate",  
  "suffix": "ly"  
}
```

Frontend Features

- **Authentication:** Sign up / log in via Supabase (magic link / email OAuth)
- **Forgot password:** Rate-limited (429 after repeated clicks), 60-second cooldown, actionable error messages

- **Reset password:** PKCE code exchange on mount; fallback "Get a fresh link" if code is expired
- **Recovery link forwarder:** Global App component redirects any route to `/reset-password` so Supabase links land in the correct app
- **Analyzer page:** Input word → get prefix/root/suffix breakdown with highlights
- **Batch mode:** Upload CSV or enter words separated by newlines
- **Comparison mode:** Upload a gold-standard CSV and score the analyzer's output
- **Dictionary viewer:** Browse the 24 prefixes and 24 suffixes used by the analyzer
- **Settings:** Profile and application preferences

Authentication & Password Reset

Supabase Configuration

- **Project ref:** `gkfwxixscktwhcnuuajt`
- **Site URL:** Must be set in Supabase Dashboard → Authentication → URL Configuration (e.g., `localhost:5173` for dev, or the production domain)
- **Redirect URLs:** Add `http://localhost:3001/reset-password` so password-reset links land in Affixa rather than MarketMind/LINEAGE

Flow

1. User clicks **Forgot password** → sends recovery link to user's email
2. Email link contains a PKCE-mediated URL that lands on the frontend
3. `App.tsx` catches the link at any route and forwards it to `/reset-password`
4. `ResetPassword.tsx` exchanges the PKCE code on mount
5. User sets a new password

Rate limiting & cooldown

- Repeated password-reset clicks within a minute trigger Supabase's 429 rate limit
- The frontend shows a 60-second cooldown timer after a request
- Error messages guide the user to wait and try again

Evaluation Results

Tested against two gold-standard CSV datasets:

Dataset	Words	Prefix Accuracy	Root Accuracy	Suffix Accuracy
gold_standard.csv	10	94.97%	76.10%	80.50%
gold_standard_extended.csv	159	(see below)		

Extended dataset summary (159 words):

- **Prefix Accuracy:** 94.97%
- **Root Accuracy:** 76.10%
- **Suffix Accuracy:** 80.50%

Per-word output is available in `backend/evaluate.py` — each line shows `word | Pred: (prefix, root, suffix) | Gold: (gold_prefix, gold_root, gold_suffix)`.

Deployment

Docker production flow

1. `docker compose up -d --build` builds and starts both containers
2. Frontend image: `affixa-frontend:latest` — Node 22 build, nginx, SPA fallback, dictionaries copied at build time
3. Backend image: `affixa-backend:latest` — Python 3.11-slim, WordNet pre-downloaded, `uvicorn` on `0.0.0.0:8000`
4. Port mapping: `3001 → 80` (frontend), `8001 → 8000` (backend)

5. `VITE_API_URL` defaults to `http://localhost:8001/api` (set via compose args)

Post-deployment checklist

- [] Verify `http://localhost:3001` loads the Affixa home page
- [] Test `http://localhost:8001/api/health` → OK
- [] Test Forgot Password flow → check inbox → open reset link in the **same tab** (App.tsx forwarder)
- [] Add `http://localhost:3001/reset-password` to Supabase → URL Configuration → Redirect URLs
- [] Test Analyzer with a word (e.g., "unfortunately")
- [] Run batch comparison if desired
- [] Confirm no port conflicts with other services on the host

Known constraints

- Host port 3000 = MarketMind AI, 5173 = LINEAGE, 8000 = WSL relay → use 3001 and 8001
- Supabase cross-origin redirects must be allowlisted in the dashboard; this is outside the repo
- Frontend imports `prefixes.json` / `suffixes.json` from the backend at build time — Dockerfile copies both trees (`COPY backend/app/nlp/dictionaries ./backend/app/nlp/dictionaries`)
- Repeated reset-clicks within a minute hit Supabase's rate limit (429) — the 60s cooldown prevents escalation

Solo Development Notes

This project is maintained independently. No teammate names or contributions appear in this documentation. All code, configuration, and deployment scripts are authored solely for the Affixa morphological analyzer.

Generated for the Affixa solo project. See `docs/DEPLOYMENT.md` for the latest deployment checklist.